



Enterprise IT Compliance & Governance Framework

Production-Grade Policy Portfolio for IT Compliance & Audit Roles

Document ID:	ECF-2026-v1.0
Author:	IT Compliance Portfolio Project
Standards Framework:	ISO/IEC 27001:2022 SOC 2 Type II (Trust Services Criteria)
Classification:	Public Portfolio Demonstration
Last Reviewed:	July 2026

POL-CM-01: Secure Software Development & Change Management Policy

Compliance Matrix Mapping:

ISO 27001:2022: Control A.8.32 (Change Management) & A.8.25 (Secure Development)

SOC 2 Type II: Common Criteria 8.1 (Change Management & Lifecycle Controls)

1. Purpose

This policy defines the mandatory controls for proposing, developing, reviewing, testing, and deploying changes to production systems and codebases to minimize operational and security risks.

2. Scope

This policy applies to all source code repositories, configurations, infrastructure-as-code (IaC), and continuous integration/continuous deployment (CI/CD) pipelines managed under the Enterprise GitHub Organization.

3. GitHub Policy-as-Code Controls (Mandatory)

3.1 Repository Rulesets & Branch Protection

All production branch deployments (e.g., `main`, `master`, `production`) must enforce GitHub Repository Rulesets with the following minimum configurations:

- **Require a pull request before merging:** Direct pushes to default branches are strictly prohibited.
- **Required approvals:** A minimum of **two (2) independent peer reviews** (Write access or higher) must approve any PR before merge.
- **Code Owner Reviews:** If changes impact sensitive directories (e.g., `/infrastructure`, `/auth`), approval must be secured from designated individuals defined in the `CODEOWNERS` file.
- **Require status checks to pass:** Prior to merging, all automated CI pipelines (linters, unit tests, vulnerability scanners) must complete successfully.
- **Dismiss stale pull request approvals:** If a branch is updated after an approval, existing approvals are automatically revoked, requiring a re-review.

- **Restrict deletions:** Branch deletion of default or protected branches is restricted to Organization Administrators.

3.2 Commit Signing

- All commits must be cryptographically signed (GPG, SSH, or S/MIME keys) to guarantee authenticity and non-repudiation.
- Unsigned commits will be rejected automatically by GitHub push protection rules.

3.3 Security Gates

- **Static Application Security Testing (SAST):** GitHub Advanced Security (CodeQL) must run on every pull request. No PR can be merged with unresolved "High" or "Critical" vulnerability alerts.
- **Secret Scanning:** Active push protection must be enabled organization-wide to block commits containing plaintext API keys, passwords, or certificates.

POL-AC-01: Identity, Authentication, and Access Control Policy

Compliance Matrix Mapping:

ISO 27001:2022: Control A.5.15 to A.5.18 (Access Control & Identity Management)

SOC 2 Type II: Common Criteria 6.1, 6.2, 6.3 (Access & Logical Identity Controls)

1. Purpose

To establish cryptographic identity verification, mandate the Principle of Least Privilege, and ensure strict credential management to protect organizational resources on GitHub.

2. Scope

Applies to all internal employees, contractors, third-party collaborators, and automated service accounts accessing the GitHub organization.

3. Mandatory Access Controls

3.1 Authentication & Multi-Factor Authentication (MFA)

- **Identity Provider (IdP) Integration:** All users must authenticate via Enterprise Single Sign-On (SSO) integrated with the company's centralized Directory (e.g., Okta, Entra ID).
- **Enforced MFA:** Multi-Factor Authentication (MFA) is strictly required for all GitHub accounts within the organization. Accounts failing to enroll in MFA within 24 hours of invitation will be automatically deprovisioned.
- **Session Lifetimes:** Idle sessions on GitHub Enterprise UI must be restricted to a maximum of 8 hours before requiring re-authentication.

3.2 Access Provisioning & Least Privilege

- **Default Permissions:** The default base permission for all organization members is set to **Read** only.
- **Role-Based Access Control (RBAC):** Repository access must be granted using GitHub Teams. Direct individual user assignments to repositories are strictly prohibited.
- **Collaborator Policy:** External, third-party collaborators may only be added to private repositories with formal approval from the Information Security Team, subject to a bi-annual access review.

3.3 Token & Credential Lifetimes

- **Personal Access Tokens (PATs):** Only fine-grained Personal Access Tokens are permitted. Classic PATs are blocked at the organization policy level.
- **Token Expiration:** No PAT may exceed an expiration window of **30 days**.
- **IP Allow List:** Access to the Enterprise Organization resources via SSH, HTTPS, or API is restricted to authorized company VPN IP ranges.

SEC-WF-01: GitHub Actions CI/CD Security Workflow

Compliance Matrix Mapping:

CIS Controls: Control 16 (Application Software Security)

Continuous Auditing: Automated Policy Linting & Verification

1. Policy Overview

This automated executable script serves as a technical continuous auditing check. It monitors documentation updates to verify styling consistency, layout, and structure alignment before merging documentation changes into the production system.

2. Executable Code Block (YAML Syntax)

File Path: `.github/workflows/compliance-gate.yml`

```
name: Compliance Verification Gate

on:
  pull_request:
    branches: [ main ]
    paths:
      - '**.md' # Trigger on policy updates

jobs:
  compliance-audit:
    name: Lint & Audit Policy Documents
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository Code
        uses: actions/checkout@v4

      - name: Lint Markdown Files for Accessibility and Standards
        uses: DavidAnson/github-markdown-lint@v1
        with:
          config: '{"default": true, "MD013": false}'

      - name: Policy Verification Status
        run: |
          echo "Compliance Verification Check Passed Successfully."
          echo "All modifications adhere to documentation governance templates."
```